PYCON US 2026 - 30 MIN

Breaking the Speed Limit

Fast statistical models with Python 3.14, Numba, and JAX - told from the workflow of a statistician.

Wenxin Jiang and Jian Yin - Department of Biostatistics, City University of Hong Kong

Trust -> speed -> scale

Prove the statistic, accelerate the bottleneck, then test scale.

Simulation contract

Controlled truth before runtime charts.

Python interface

Numba and JAX move the hotspot, not the scientific contract.

WHY THIS TALK

For statisticians, speed is useful only after trust

Optimization enters after reference behavior is defined, stressed, and checked.

Statistical specification evolves

Simulation exposes assumptions, hard cases, and stable outputs.

Reference code is scientific specification

Readable Python or NumPy is the executable contract.

Acceleration is conditional

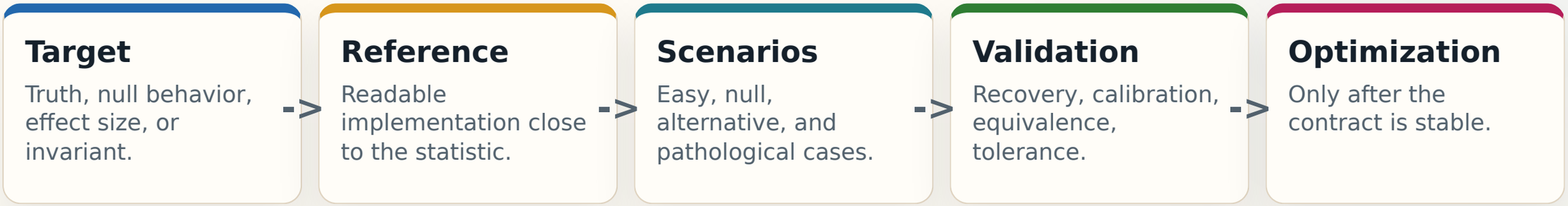
Faster paths must compute the same statistic on controlled scenarios.

A faster implementation that changes the statistic is a different method.

SIMULATION AS TESTING

Simulation creates an answer key

Real data rarely tells us the truth; simulation lets us control part of it.



PROGRAMMER BRIDGE

Simulation-driven statistical computing is statistical CI

The testing instinct is familiar; the oracle is statistical behavior rather than one expected scalar.

Unit tests	Fixed-seed equivalence to the reference statistic.
Property tests	Null invariants, symmetry, monotonicity, calibration.
Stress tests	Imbalance, outliers, bad seeds, high dimension.
Load tests	Scale N, d, K, p, R, workers, and batch sizes.
Regression tests	Optimized paths keep preserving the same statistic.

WHY PYTHON

Python is the control plane; the hotspot can move to the right engine

Keep the model readable, then move the measured hotspot into the smallest engine that preserves the simulation contract.

Python / NumPy reference
Executable statistical definition.

->

Simulation contract
Recovery, calibration, equivalence, and tolerance gates.

->

Hotspot shape
Loops, dense algebra, repetition, or batched arrays.

NumPy / BLAS
Dense algebra and matrix identities.

Numba
CPU numerical loops without giant temporaries.

Threads / Python 3.14
Repeated work over shared arrays.

JAX / A100
Batched array programs after reformulation.

WHY THESE TWO EXAMPLES

Two workloads, two statistical computing shapes

ITERATIVE MODEL FITTING

k-means

Assignment -> update -> repeat.

Pressure: loops, distances, temporary arrays, iteration dependence.

RESAMPLING INFERENCE

Permutation test

Shuffle -> statistic -> repeat.

Pressure: random generation, batching, memory layout, independent draws.

They stand for two common shapes: iterative dependence and repeated simulation.

EVIDENCE LADDER

Evidence ladder: trust, scale, acceleration

MacBook Air

Trust tier: controlled simulation and reference equivalence.

->

Linux server CPU

Scale tier: larger shapes, parallelism, and memory accounting.

->

A100 GPU

Accelerator tier: only when the pipeline is batched and device-resident.

Correctness before runtime

Every timing row carries a validation status.

Warm timing plus memory

Cold calls, warm medians, and RSS are logged separately.

Unavailable is data

JIT, GPU sync, and interpreter state are explicitly labeled.

WORKLOAD 1

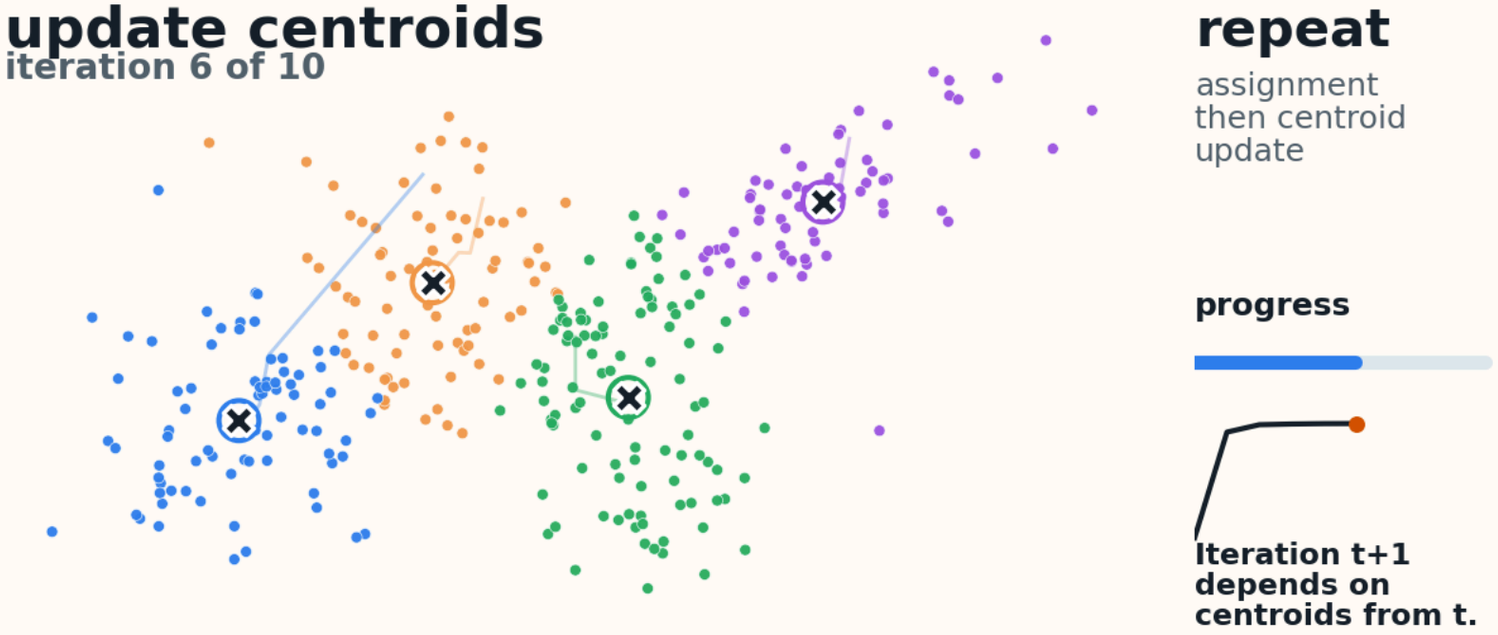
k-means as iterative model-fitting pressure

Same data and initialization; vary geometry and scale.

METHOD VISUALIZATION

How k-means moves

Assignment and centroid update form an iterative dependency loop.



One Lloyd loop

1. assign each point to the nearest centroid
2. update centroids from assigned points
3. repeat until stable

Each iteration depends on the previous centroids.

This is why explicit loops, temporary arrays, and distance computation connect naturally to Numba, NumPy matmul, and JAX/A100.

WHY K-MEANS WORKS HERE

A simple algorithm exposes common iterative pressure

Statistical contract

Recover known clusters and preserve fixed-initialization inertia.

Failure modes

Low separation, imbalance, outliers, bad starts, empty clusters.

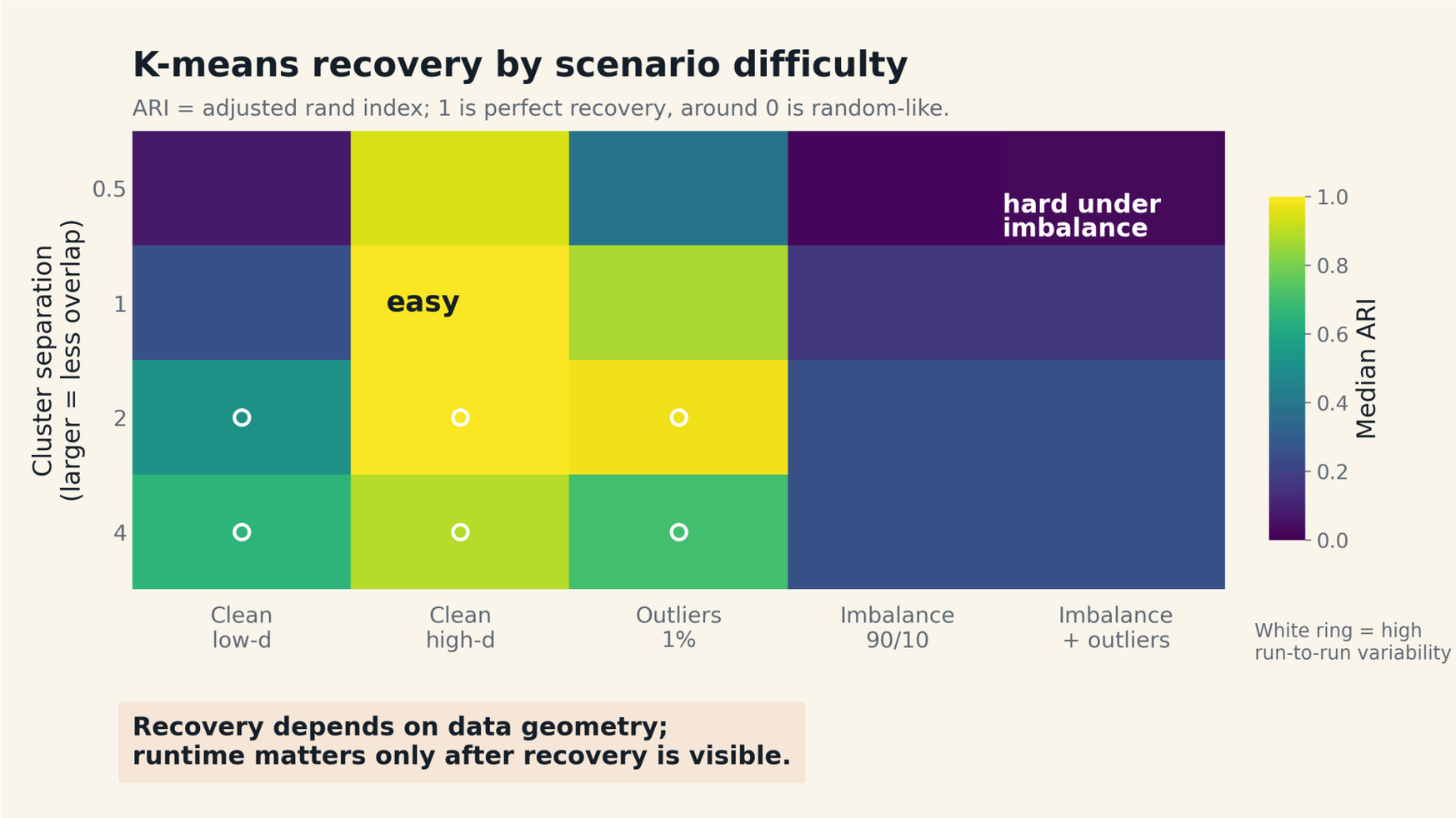
Compute pressure

Assignment/update loops, distances, temporaries, repeated iterations.

Numba loops **NumPy matmul distance algebra** **JAX/A100 large regular loops**

SIMULATION RESULT BEFORE SPEED RESULT

Recovery comes before runtime



ARI = 1 means perfect recovery; hard geometry makes runtime-only comparisons misleading.

REFERENCE GATE

Optimized paths must preserve the reference statistic

Same initialization, same reference solution, and explicit tolerance before timing.

PASS

Memory-risk skips are expected safeguards, not optimized failures.

3,840
checked implementation rows

0
optimized failures

480
expected memory-risk skips

3.1e-14
max relative difference; tolerance 1e-8

SCALE TIER

Server k-means: acceleration is shape-dependent

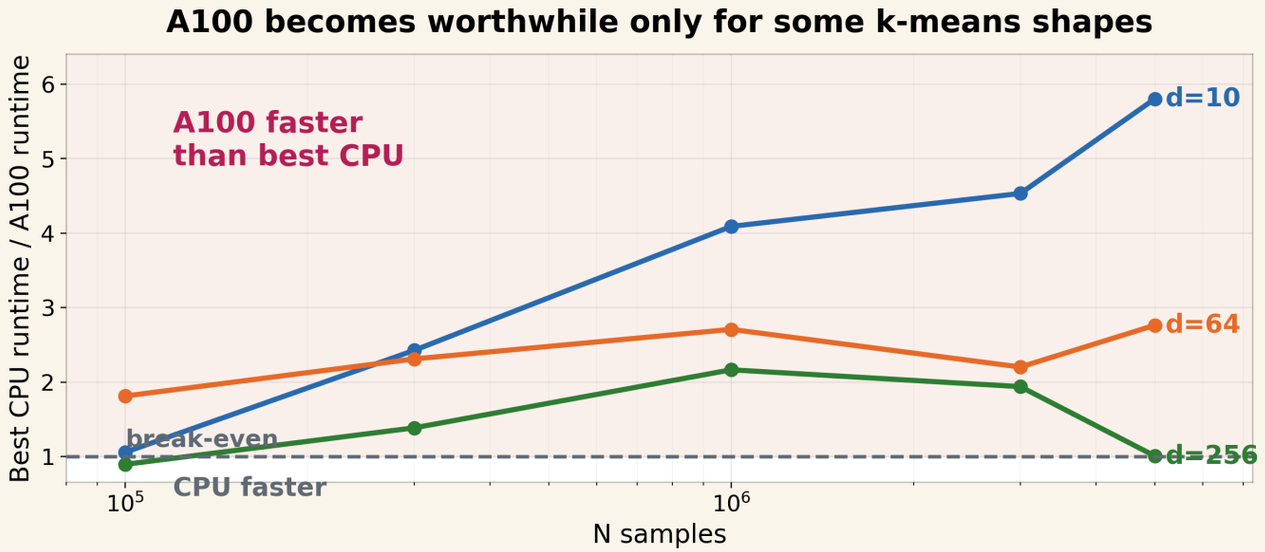
Speedup definition
Best matched CPU baseline / A100 warm runtime.

Shape matters
Numba, BLAS, and A100 win in different regions.

Caution
High dimension can narrow A100 advantage.

Server k-means: best CPU baseline vs A100

k=20, separation=2.0; maximum observed A100 advantage 5.8x.



A100 helps only after enough regular work exists.

Measured server rows only: k=20 and separation=2.0; max matched A100 edge 5.8x.

K-MEANS TAKEAWAY

The tool choice follows the simulation signal

Unstable recovery

Do not optimize yet; the statistical contract is not ready.

Scalar loops

Use Numba to compile explicit CPU kernels.

Dense distance algebra

Use NumPy/BLAS when the rewrite removes large temporaries.

Huge regular work

Use JAX/A100 only after enough batched work exists.

k-means stands in for a large class of iterative statistical algorithms: EM, coordinate descent, optimization loops, and simulation-based estimators.

WORKLOAD 2

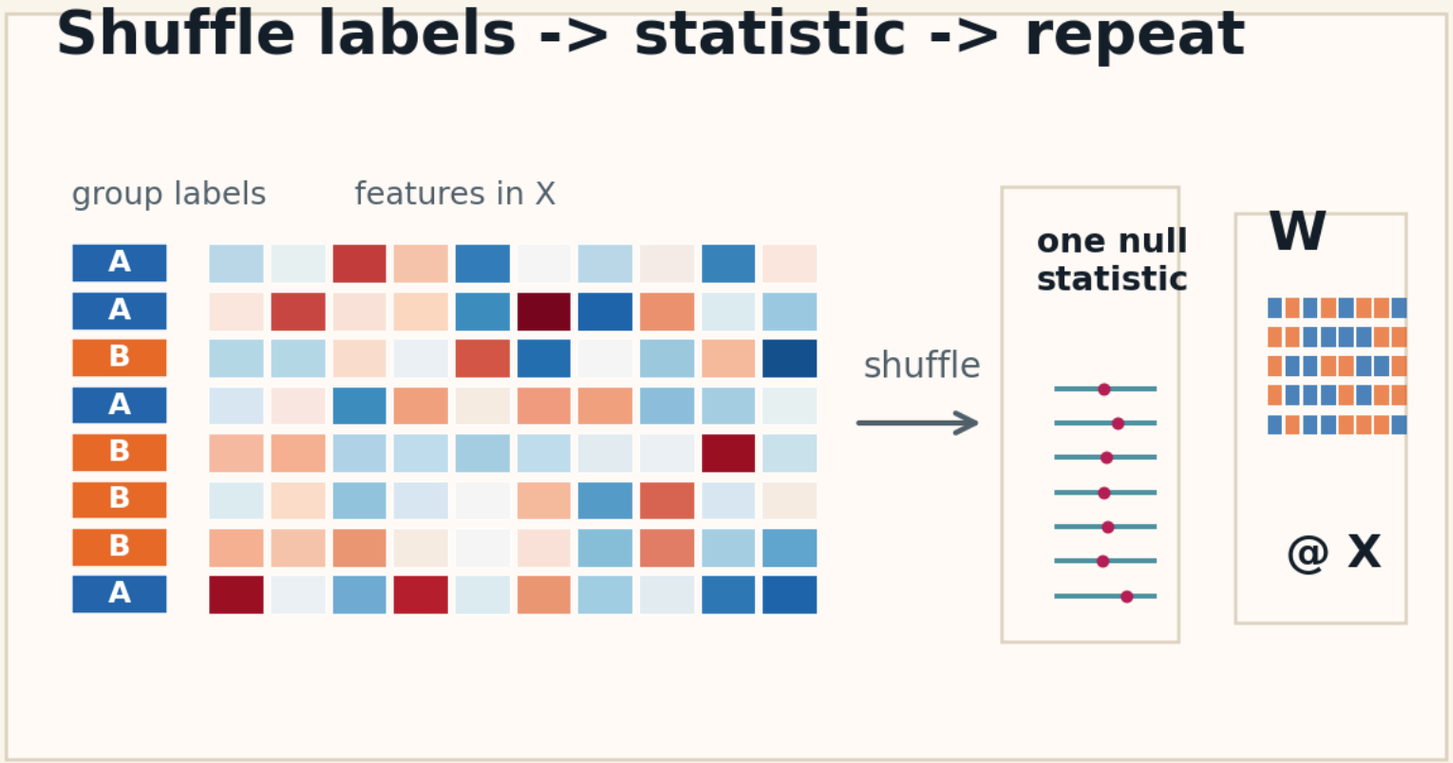
Permutation tests as resampling inference pressure

A simple inference procedure becomes expensive when we need thousands of null draws across a high-dimensional features matrix.

METHOD VISUALIZATION

How a permutation test scales

One simple statistic becomes many independent null draws.



One resampling loop

1. shuffle group labels
2. recompute the statistic
3. repeat many times to form the null distribution

The repetitions are independent; the pipeline is not free.

This is why permutation tests connect to threads/processes, Numba kernels, and JAX/A100 matrix formulations.

WHY PERMUTATION TESTS WORK HERE

X = samples x features is the visual anchor

Data shape

Samples by features: genes, biomarkers, voxels, or other high-dimensional measurements.

Statistical contract

Null p-values stay calibrated; optimized p-values match the reference under the same permutations.

Compute pressure

Random labels, shared arrays, batching, memory layout, and transfer overhead.

Independent repetitions still have a pipeline cost.

CODE SHAPE

Statistical algebra changes the kernel, not the statistic

Readable reference loop

```
for perm in permutations:
    t_null.append(statistic(X, perm))
```

Matrix formulation

```
W = contrasts(permutations)
t_null = W @ X
```

Only valid after same-permutation equivalence checks.

CORRECTNESS GATE 1

First, prove the matrix path matches the reference

Same permutation stream; same p-values; only floating-point-scale statistic gaps.

Same permutations

Reference loop and matrix path use the same label draws.

Same p-values

max |p diff| = 0.0 across checked rows.

Floating-point statistic gap

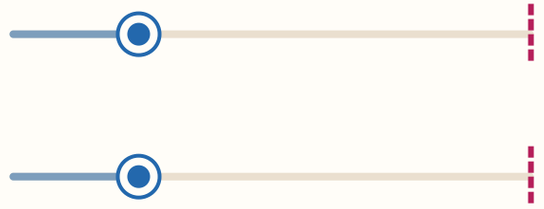
max |stat diff| = 9.4e-16, far below tolerance 1e-6.

PASS

18 workloads × 5 seeds
same permutation stream
JAX tier: CPU/x64

450	0	45
pass rows admitted to comparison	failed rows	expected memory-risk skips

Nonzero statistic differences only; tolerance 1e-6



Same statistic, different computational shape.

CORRECTNESS GATE 2

Null calibration gate: type-I error stays near alpha

Under the null, the type-I error should stay close to nominal alpha = 0.05.

Why calibration matters

Equivalence checks implementation.
Calibration checks statistical behavior.

Observed check

Observed type-I error = 0.051 at nominal alpha 0.050.

What this unlocks

Now runtime comparisons become meaningful.

PASS

0.051

observed type-I error estimate

0.050	100	500	1,000
nominal alpha	null replicates	n	p and R

Calibration ruler

Dashed: nominal 0.05 **Dot:** observed mean 0.051

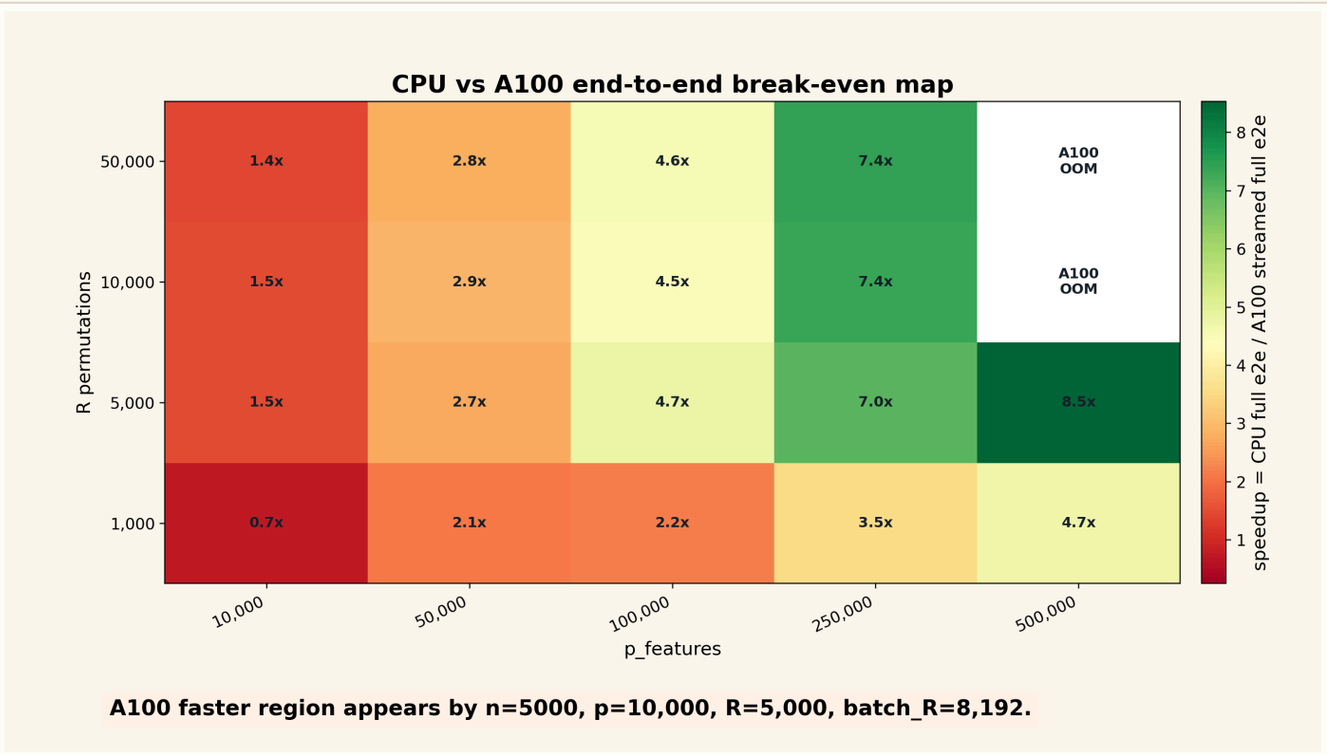
Expected band: 0.05 +/- 1.96 * sqrt(0.05 * 0.95 / 1000).

Fast code is useful only if null calibration remains correct.

DECISION MAP

When does GPU help for permutation tests?

A100 wins only after the permutation pipeline becomes large, batched, and device-resident enough.



speedup = matched CPU matrix baseline / A100 streamed full end-to-end.

Timing scope

Compile excluded; transfer included; kernel-only excluded.

CPU wins when

p and R are small or random generation, W construction, transfer, collection, or too-small batch_R dominates.

A100 can win when

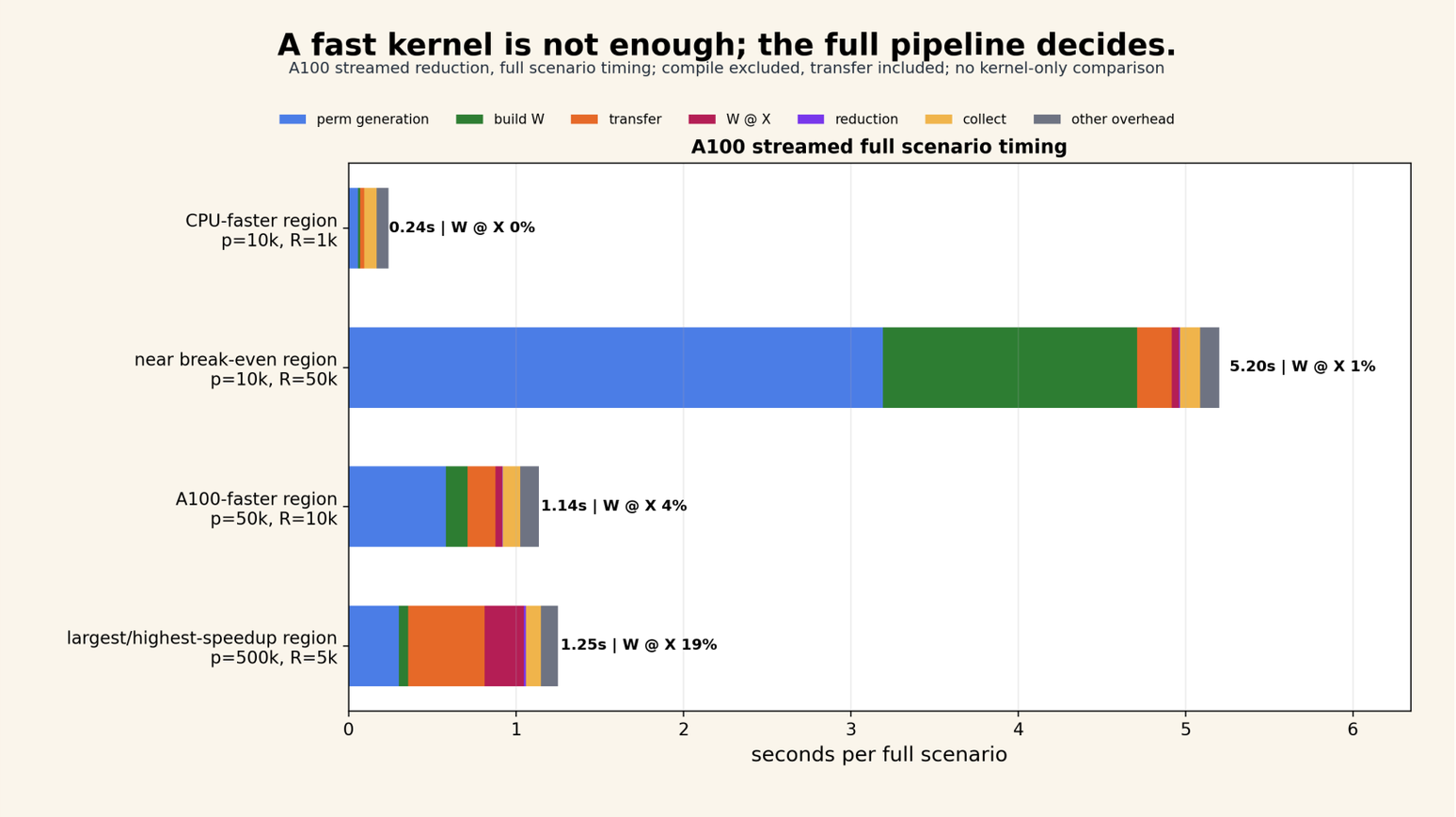
R x p is large, batch_R fits memory, streamed reduction avoids collecting R x p, and work stays on device.

Measured boundary: A100 first wins at n=5,000, p=10,000, R=5,000, batch_R=8,192; use CPU when overhead dominates.

PIPELINE DECOMPOSITION

Where does A100 time go?

Full-scenario streamed path; compile excluded, transfer included.



A fast kernel is not enough; the full pipeline decides.

PARALLELISM

CPU parallelism must be measured under resource constraints

Linux server CPU, controlled 1/4/16/64/128 sweep; high counts are shared-server evidence.

k-means

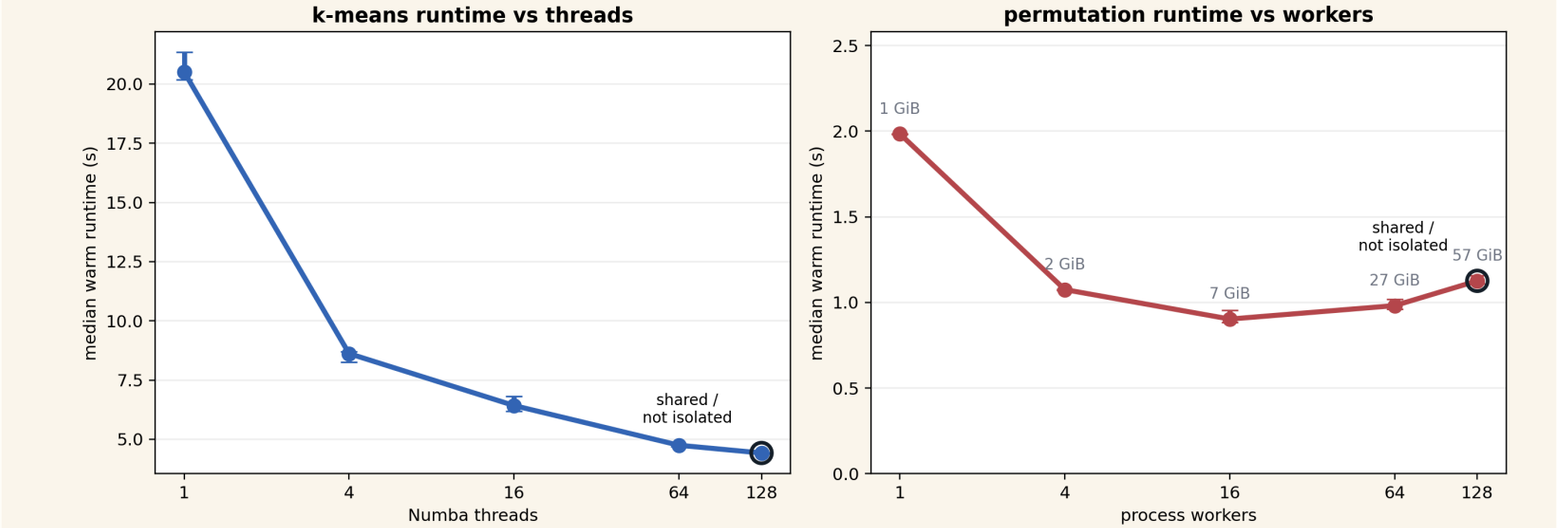
Numba kept improving through 128 in this shared-server run.

permutation

Best runtime was at 16; memory rose to about 58 GiB at 128.

128 caveat

Affinity: 512 CPUs; no exclusive allocation.



High worker counts on a shared server are hard to interpret without CPU affinity and load checks.

CONNECTING THE TOOLS

Python 3.14, Numba, and JAX solve different parts of the workflow

Python 3.14 = runtime ceiling

Threads and free-threaded builds can change repeated-work ceilings; measure availability.

Numba = CPU loop kernel

Compile explicit numerical loops when temporaries or interpreter overhead dominate.

JAX/A100 = batched array program

Use after reformulation makes device-resident work dominate the pipeline.

Choose runtime after simulation identifies the bottleneck.

AI CODING TOOLS

Codex can generate variants; the statistician owns the contract

Automation layer

- Generate implementation variants.
- Run scenario grids.
- Record environment metadata.
- Regenerate plots from CSVs.

Statistical contract

- Define the statistic and null model.
- Choose failure criteria.
- Interpret calibration and recovery.
- Decide what evidence is enough.

AI accelerates experiment hygiene; it does not define statistical truth.

DECISION GUIDE

Choose the smallest tool that preserves the statistic

Can I trust the result?

Python/NumPy reference

No: make the statistic testable first.

Numba

Scalar numerical loops on CPU.

Threads / free-threaded Python

Repeated work over shared arrays.

JAX/GPU

Large batched array program.

Algebraic rewrite

Giant temporaries or better kernel shape.

multiprocessing

Process-isolated jobs with memory budget.

CLOSE

Make the statistic testable. Then make the bottleneck fast.

Clear enough to trust. Fast enough to scale.

`github.com/lucajiang/FastStatisticalModels4Python`

BACKUP

Evidence map: keep tiers separate

MacBook Air trust tier

k-means equivalence, recovery surfaces, permutation p-value equivalence, calibration, and power.

Linux server CPU scale tier

k-means shape scaling, Numba thread sweep, permutation R/p scaling, worker and memory sweeps.

A100 accelerator tier

JAX k-means break-even, old permutation matched-slice negative evidence, and the streamed-reduction break-even follow-up.

3,840
k-means local pass rows

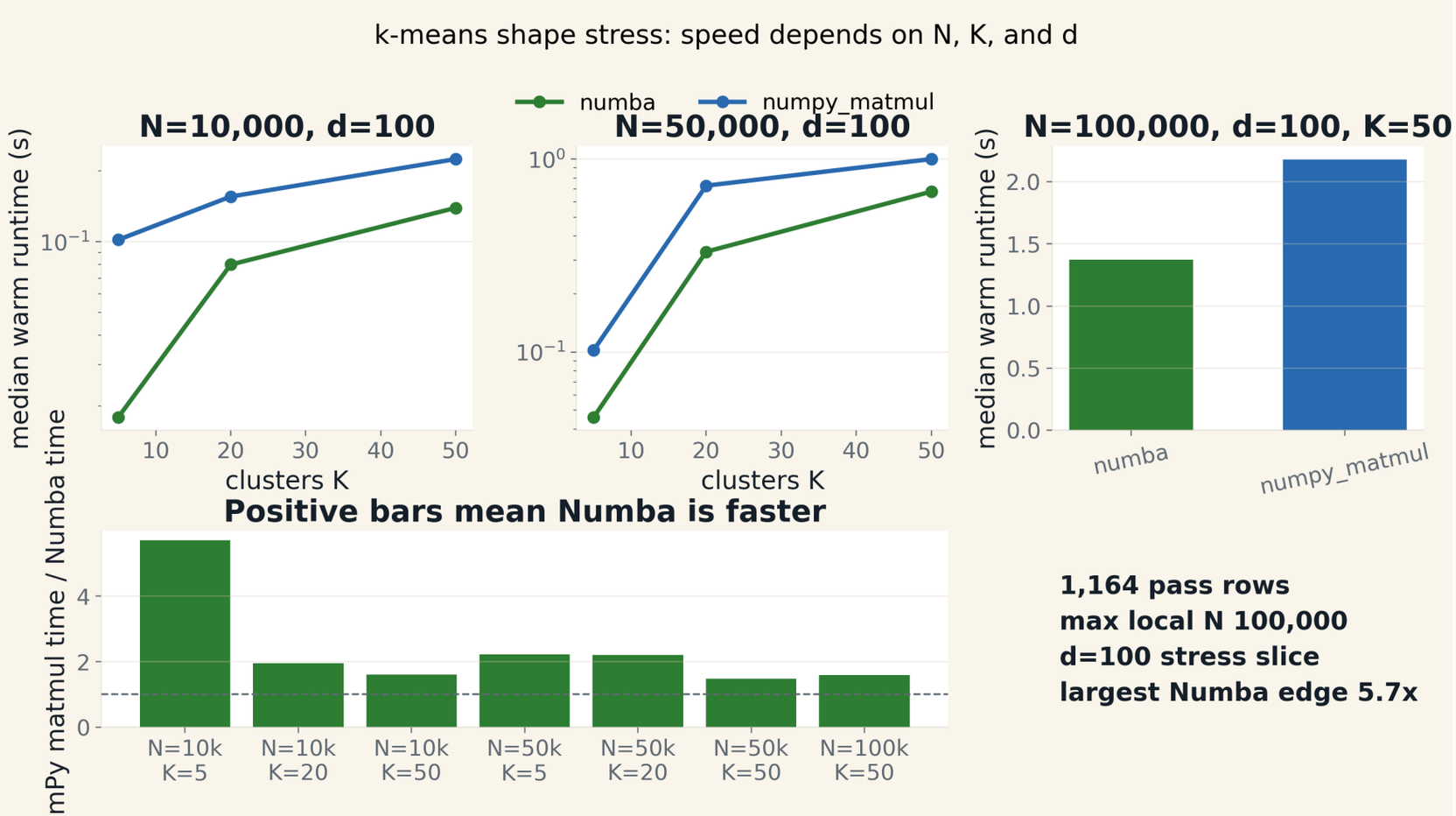
450
permutation equivalence pass rows

540
server CPU k-means pass rows

135
A100 k-means pass rows

BACKUP

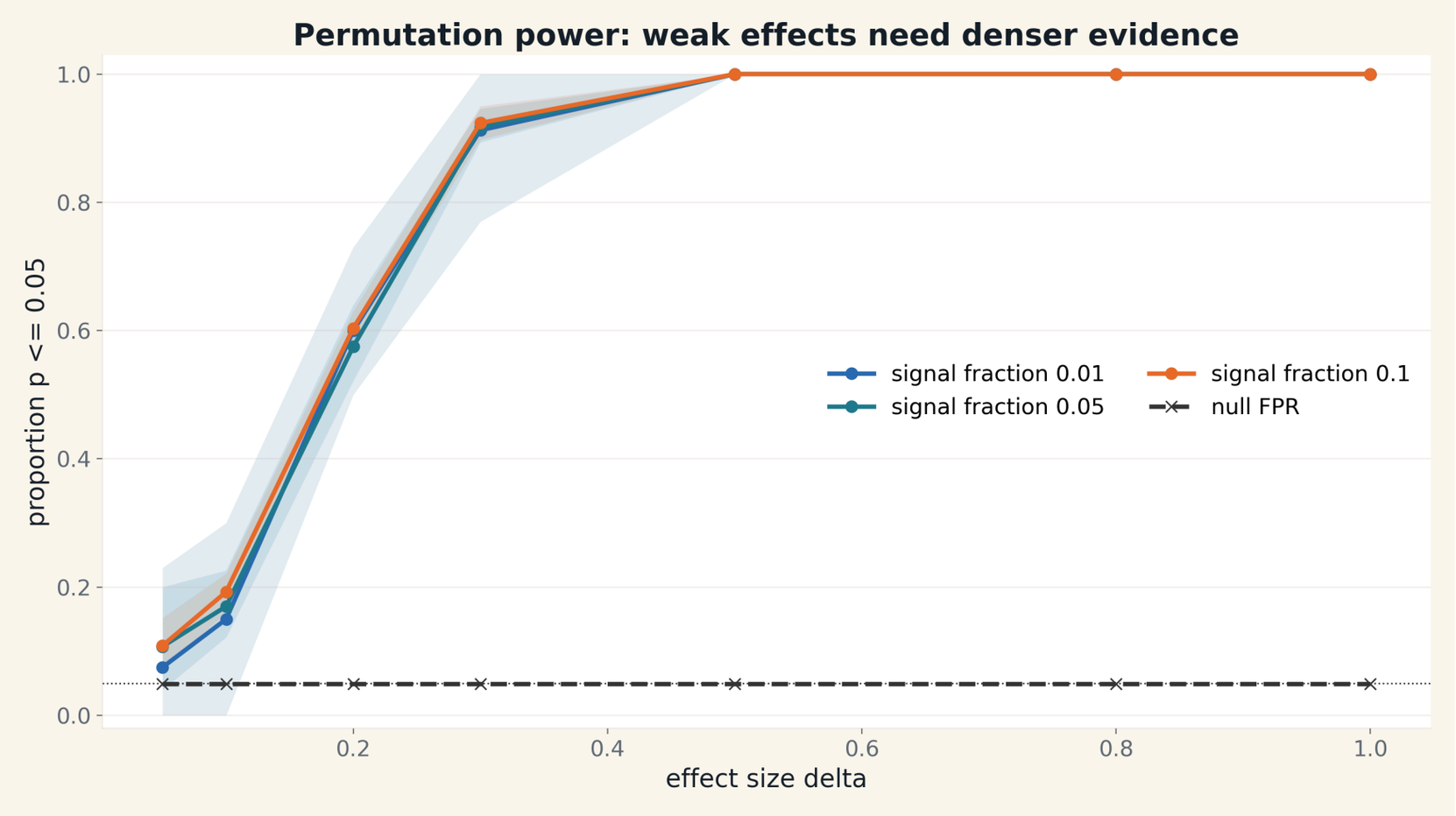
Shape stress: K and d become the bottleneck



Extra MacBook evidence: 1,164 shape-stress rows, all pass.

BACKUP

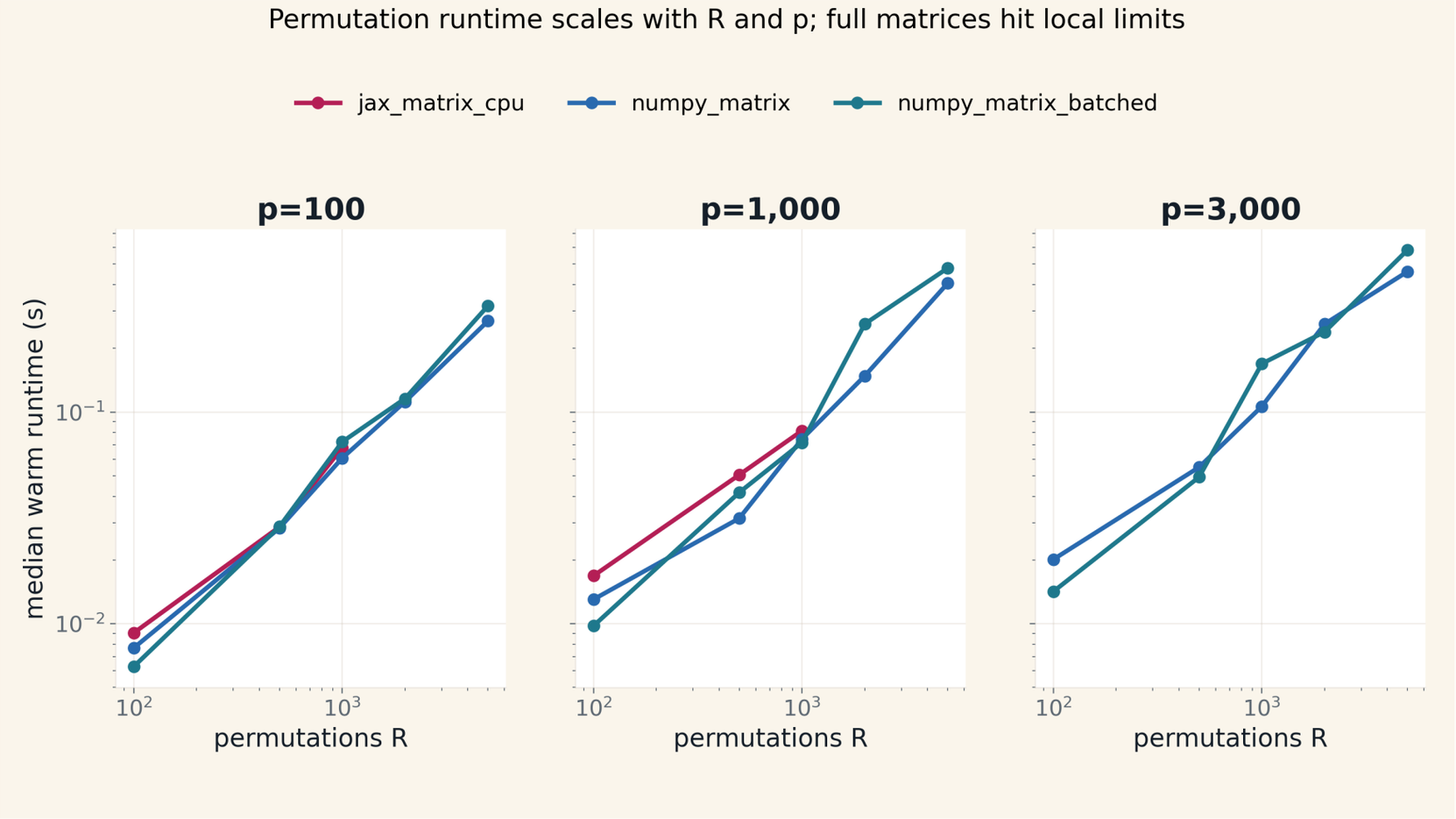
Power increases with effect size



Backup statistical sanity check: weak effects need denser evidence; larger effects reach high power in this local curve.

BACKUP

MacBook-only permutation runtime follows computational shape



MacBook CPU only: useful for local validation, not a server or GPU leaderboard.

BACKUP

What the coding agent automated

Experiment surface

Replaced quick outputs with resumable MacBook evidence, server long-safe CSVs, and result READMEs.

Visualization surface

Regenerated figures from CSVs and removed charts that looked precise but were unreadable.

Repository surface

Updated documentation so future agents land on the current simulation contract.